PROCESADORES DEL LENGUAJE

GRADO EN INGENIERÍA INFORMÁTICA

MEMORIA DE LA PRÁCTICA

Práctica 3 - Análisis semántico de Lava

Incluye generación opcional de código intermedio

Curso 2025/2026

DATOS DE LOS AUTORES

Nombre:	Marco Garcia
NIA:	100525997
Correo electrónico:	100525997@alumnos.uc3m.es
Grupo de clase:	80
Titulación cursada:	Grado en Ingeniería Informática

Nombre:	Pablo Roig
NIA:	100525980
Correo electrónico:	100525980@alumnos.uc3m.es
Grupo de clase:	80
Titulación cursada:	Grado en Ingeniería Informática

Índice

1. Introducción y objetivos
2. Estructura general de la solución
3. Integración con P1 y P2
4. Modelo de análisis semántico
5. Tablas de símbolos, registros y funciones
6. Sistema de tipos y conversiones
7. Validación de expresiones, sentencias y control de flujo
8. Tratamiento de errores y modo de ejecución
9. Generación opcional de código intermedio
10. Batería de pruebas
11. Decisiones de diseño, limitaciones y conclusiones

Resumen
La P3 completa el compilador Lava con una fase semántica sobre el arbol estructural producido por el parser. Se implementan ámbitos, tablas de registros y funciones, resolución de sobrecarga, comprobacion de tipos, promocion int a float, validación de constructores, acceso a campos, control de return/break y una extensión opcional para emitir código intermedio de tres direcciones simplificado.

1. Introducción y objetivos

Esta memoria documenta la tercera entrega de la práctica de Procesadores del Lenguaje, centrada en ampliar el lexer y parser desarrollados en P1 y P2 con una fase de análisis semántico para el lenguaje Lava. El objetivo principal es que el programa no se limite a reconocer una secuencia sintácticamente valida, sino que también compruebe si las construcciones tienen sentido según las reglas de tipos, declaraciones, ámbitos y uso de funciones definidas en el enunciado.

El analizador acepta programas válidos sin producir salida y devuelve mensajes de error cuando detecta incumplimientos semánticos. Esta decisión mantiene el comportamiento solicitado para P2 y lo extiende con una validación más profunda: variables declaradas antes de usarse, asignaciones compatibles, condiciones booleanas, constructores de registros correctos, resolución de llamadas a función y control estricto de return y break.

Además, se ha incorporado el apartado opcional de generación de código intermedio. Este modo no sustituye al análisis semántico: primero se parsea el programa y se comprueba que no contiene errores; solo entonces se emite un fichero .ir. De este modo se evita generar una representación intermedia de un programa que ya se sabe incorrecto.

2. Estructura general de la solución

La entrega se organiza en los siguientes módulos:

Módulo	Responsabilidad principal
lexer.py	Analizador léxico heredado de P1. Reconoce tokens, comentarios, literales y palabras reservadas.
parser.py	Analizador sintáctico de P2. Construye nodos estructurales con kind, line y campos especificos de cada produccion.
semantic.py	Nueva fase de P3. Contiene Scope y SemanticAnalyzer, las tablas globales y todas las comprobaciones semánticas.
ir.py	Generador opcional de código intermedio. Recorre los nodos ya validados y emite temporales, etiquetas, saltos y llamadas.
main.py	Punto de entrada. Gestiona los modos normal, --token y --ir.
tests/	Batería automatizada de casos válidos, inválidos y muestra IR.

El flujo completo es: entrada .lava, lexer, parser, representación estructural, analizador semántico y, opcionalmente, generación IR. La fase semántica queda desacoplada del parser para mantener la gramática clara y para poder aplicar reglas dependientes de contexto sin forzar producciones artificiales.

3. Integración con P1 y P2

La P3 reutiliza el lexer de P1 y el parser de P2 como base estable. Esta continuidad es importante porque el enunciado plantea la práctica como un compilador incremental: primero se reconocen los elementos léxicos, después la estructura gramatical y finalmente la validez semántica del programa.

- **Lexer: se conservan los tokens de Lava, incluyendo tipos primitivos, literales, operadores aritméticos/lógicos/comparativos, delimitadores, record, new, print, break, return e identificadores.**
- **Parser: se mantiene la gramática LALR(1), la precedencia de operadores y la separación entre primitive_type y custom_type para imponer la inicialización obligatoria de registros.**
- **Nodos: se aprovecha la función node(kind, line, **data), que permite mantener una representación uniforme y suficiente para recorrer el programa sin depender de detalles internos de PLY.**

```
def node(kind, line, **data):
    value = {"kind": kind, "line": line}
    value.update(data)
    return value
```

La memoria de P2 ya justificaba que ciertas reglas de contexto se pospusieran a P3. Esta entrega cierra esa decisión: break, return, existencia de tipos, compatibilidad de operadores y resolución de funciones se comprueban en semantic.py, donde si existe información de ámbitos y tipos.

4. Modelo de análisis semántico

El analizador semántico se implementa mediante la clase SemanticAnalyzer. La estrategia sigue tres pasos: primero se recolectan registros y funciones globales; después se analizan las declaraciones y sentencias de nivel superior; por último se recorren los cuerpos de las funciones con sus parámetros y ámbitos locales.

```
def analyze(self, program):
    self.collect_records_and_functions(program)
    self.analyze_top_level(program)
    self.analyze_functions(program)
    return self.errors
```

La recolección inicial permite que una llamada a función pueda resolverse aunque la declaración aparezca más adelante en el fichero. En cambio, para los registros se respeta la restricción de uso antes de declaración en los constructores new, porque el enunciado indica que los campos de registro pueden ser de otro registro previamente definido.

5. Tablas de símbolos, registros y funciones

5.1. Ámbitos y tabla de símbolos

La clase Scope modela una tabla de símbolos con enlace al ámbito padre. Cada bloque puede crear un nuevo Scope, lo que permite declarar variables locales sin contaminar el ámbito global y consultar identificadores de forma ascendente cuando no estan en el bloque actual.

Operación	**Comportamiento**
declare(name, type_name, line, errors)	Inserta un simbolo si no existe ya en el mismo ámbito. Si existe, registra un error de redeclaración.
lookup(name)	Busca primero en el ámbito actual y, si no aparece, continua por los padres hasta llegar al global.
global_names	Evita que registros, variables globales y funciones colisionen entre categorías, salvo sobrecarga de funciones.

5.2. Registros

Los registros se almacenan en self.records con su línea de declaración, la lista ordenada de campos y un mapa por nombre de campo. Durante la recogida se comprueba que no exista otro registro con el mismo nombre, que no haya campos repetidos y que los tipos de los campos existan.

5.3. Funciones y sobrecarga

Las funciones se guardan por nombre en self.functions, donde cada entrada contiene varias firmas posibles. La firma relevante es la tupla de tipos de parámetros, lo que permite sobrecarga cuando cambia la aridad o cambia al menos uno de los tipos. Si se repite exactamente la misma firma, se informa de función duplicada.

Decision de diseño
Se permite que varias funciones compartan nombre, pero no que una variable global o un registro reutilice ese mismo identificador. Esto evita ambiguedades entre categorías y mantiene limpia la resolución semántica.

6. Sistema de tipos y conversiones

El sistema de tipos distingue entre tipos primitivos, tipos numéricos y tipos de registro. Los primitivos son int, float, char y boolean. Los numéricos son int y float. Los registros son nominales: dos registros solo son compatibles si comparten exactamente el mismo nombre de tipo.

Regla	Implementación
Asignacion exacta	target_type == source_type siempre es compatible.
Promocion numerica	int puede asignarse a float de forma automática.
Conversion inversa	float no se convierte automáticamente a int.
char y boolean	Solo son compatibles consigo mismos.
Registros	La compatibilidad es nominal, no estructural.

```
def can_assign(self, target_type, source_type):
    if target_type is None or source_type is None:
        return True
    if target_type == source_type:
        return True
    return target_type == "float" and source_type == "int"
```

La función can_assign centraliza la compatibilidad y evita duplicar lógica en declaraciones, asignaciones, argumentos de constructor, llamadas a funciones y sentencias return. Cuando un tipo es None, se evita generar errores en cascada: el error principal ya se habra registrado en la expresión que no pudo tiparse.

7. Validación de expresiones y sentencias

El metodo analyze_expression resuelve recursivamente el tipo de cada nodo. Los literales devuelven su tipo directo; los identificadores consultan la tabla de símbolos; new valida el registro y sus argumentos; field comprueba que la base sea un registro y que el campo exista; call resuelve la sobrecarga; unary y binary validan operadores.

Construccion	Comprobación semántica
Aritmética + - * /	Ambos operandos deben ser numéricos.
Comparación > >= < <=	Operandos numéricos y resultado boolean.
Igualdad ==	Numericos entre si, char con char o boolean con boolean.
Lógicos && ||	Operandos booleanos.
Unario !	Operando boolean.
Acceso a campo	La base debe ser un registro con ese campo.

7.1. Declaraciones, asignaciones y lvalues

En las declaraciones se comprueba que el tipo declarado exista y, si hay inicialización, que el valor sea asignable al tipo de la variable. En las asignaciones se tipa por separado el lado izquierdo y la expresión derecha. El lado izquierdo solo puede ser un identificador o una cadena de accesos a campos; una llamada a función, un literal o un new no son destinos válidos.

7.2. Control de flujo

Las condiciones de if, while y do-while deben ser booleanas. Para break se arrastra un indicador in_loop durante el recorrido; si aparece fuera de un bucle se emite error. Cada bloque crea un nuevo ámbito salvo el cuerpo raíz de una función, donde los parámetros ya se han declarado previamente.

7.3. Return

La implementacion adopta la regla estricta del enunciado: en funciones void no se permite return, ni siquiera sin expresión. En funciones no void se requiere al menos un return con expresión compatible con el tipo declarado. Para ello se mantiene un estado has_value_return durante el análisis del cuerpo de cada función.

7.4. Constructores new

La validación de new comprueba cuatro condiciones: que el registro exista, que no se use antes de su declaración, que el número de argumentos coincida con el número de campos y que cada argumento sea asignable al tipo del campo correspondiente. Esto cubre tanto registros simples como registros anidados.

8. Tratamiento de errores y modos de ejecución

Los errores semánticos se acumulan en una lista y se devuelven al final. El formato usado es [ERROR] mensaje en la línea X, coherente con los errores sintácticos de P2. Esta acumulacion permite informar de varios problemas en una sola ejecución cuando no dependen unos de otros.

```
python3 main.py <fichero.lava>       # lexer + parser + semántica
python3 main.py --token <fichero>    # modo léxico heredado de P1
python3 main.py --ir <fichero.lava>  # validación completa + fichero .ir
```

El modo normal no imprime nada si el programa es correcto y termina con código 0. Si hay errores, se muestran por pantalla y el proceso termina con código 1. El modo --ir sigue el mismo criterio, pero solo escribe el archivo de salida cuando la fase semántica no ha encontrado errores.

9. Generación opcional de código intermedio

Como apartado opcional de P3 se ha implementado un generador de código intermedio en ir.py. El objetivo es producir una representación textual simple, cercana a tres direcciones, que permita observar el flujo de ejecución del programa sin entrar aun en generación de código máquina.

El generador recorre los mismos nodos que el analizador semántico, pero asume que el programa ya ha sido validado. Para expresiones complejas crea temporales t1, t2, etc. Para estructuras de control crea etiquetas L1, L2, etc. Para break mantiene una pila de etiquetas de fin de bucle, de forma que un break dentro de bucles anidados salta al final del bucle correcto.

```
while (i < 3) { i = i + 1; }


L1:
t1 = i < 3
ifFalse t1 goto L2
t2 = i + 1
i = t2
goto L1
L2:
```

Elemento Lava	Salida IR
Declaracion	declare <tipo> <nombre>
Asignacion	<destino> = <valor>
Funcion	function f(params) -> tipo ... end function f
Llamada	tN = call f(args)
if	ifFalse condicion goto etiqueta_else
while/do-while	etiquetas, saltos condicionales y goto
new	tN = new Registro(args)

10. Batería de pruebas

La entrega incluye una bateria automatizada en tests/run_tests.py. Los casos se dividen en programas válidos, programas inválidos y una prueba especifica de generación IR. La ejecución realizada para esta memoria ha dado el resultado esperado: Summary: 69/69 passed.

Categoría	Cobertura
Validos	Programas vacíos, declaraciones, registros, control de flujo, funciones, expresiones complejas, sobrecarga y promocion numerica.
Inválidos sintácticos heredados	Asignaciones mal formadas, multideclaraciones con asignación, declaraciones de registro sin inicialización y EOF inesperado.
Inválidos semánticos	Variables no declaradas, redeclaraciones, tipos incompatibles, condiciones no booleanas, returns incorrectos y break fuera de bucle.
Registros	Campos inexistentes, acceso sobre primitivos, constructores con aridad o tipos incorrectos y uso antes de declaración.
Funciones	Llamadas no declaradas, argumentos incompatibles, firmas duplicadas, parámetros repetidos y conflictos de nombres globales.
IR	Generación opcional solo tras validación semántica correcta.

```
cd P3_MARCO_GARCIA_PABLO_ROIG/tests
python3 run_tests.py
Summary: 69/69 passed
```

La bateria no solo comprueba que los programas válidos pasen, sino también que los inválidos sean rechazados con al menos un mensaje [ERROR]. Esto es importante porque evita falsos positivos: un analizador semántico que no detectase un caso incorrecto sería tan problematico como uno que rechazase programas correctos.

11. Decisiones de diseño y conclusiones

La decisión principal ha sido mantener separadas las responsabilidades de parser y analizador semántico. El parser conserva una gramática manejable y produce nodos con información de línea; semantic.py se encarga de las reglas dependientes de contexto. Esta separación facilita justificar el código, ampliar comprobaciones y depurar errores sin mezclar niveles.

- Se ha centralizado la compatibilidad de tipos en can_assign para que declaraciones, asignaciones, llamadas y returns sigan exactamente la misma regla.
- Se ha implementado sobrecarga por firma, el caso más completo indicado por el enunciado para funciones.

- Se ha elegido tipado nominal para registros porque los records de Lava se declaran como tipos con nombre, no como estructuras anonimas equivalentes.
- La generación IR se ejecuta solo después de validar el programa, evitando salidas intermedias inconsistentes.

Como limitacion consciente, no se intenta hacer análisis de caminos para demostrar que todos los recorridos de una función no void retornan; se exige al menos un return con valor compatible, que es el criterio ya usado y cubierto por las pruebas de esta entrega. Tampoco se realiza recuperacion avanzada de errores, ya que el apartado opcional elegido ha sido la generación de código intermedio.

En conjunto, la P3 completa una cadena coherente de compilación para Lava: el lexer reconoce terminales, el parser valida la forma del programa, el analizador semántico verifica tipos y ámbitos, y el modo opcional --ir produce una representación intermedia útil para futuras fases.